Imagine now that Scruffy informs you there are not 1,500 competitors as anticipated but actually 150,000 competitors. He also says that he's using the program you wrote, and it takes a long time to sort the 150,000 competitors. The application freezes for several seconds each time it loads data. You need to figure out what's going on.

Timing programs

A crude but effective way to get a simple measurement of the time it takes for part of a program to execute is by using the built-in `Date` objects. To evaluate the time difference between two `Date` objects, you use subtraction:

```
begin = new Date
end = new Date
difference = begin - end
```

Create two objects from Date class

Subtract earlier date from later date

With this, you can change the `loadData` function from listing 9.1 to show how long it takes to load data:

```
loadData = ->
  start = new Date()
  console.log 'Loading and processing data'
  sortedCompetitorsFromFile fileName, (data) ->
    elapsed = new Date() - start
    console.log "Data loaded in #{elapsed/1000} seconds"
    server data
```

It takes less than one-tenth of a second for 1,500 competitors, but for 150,000 competitors it takes 35 seconds! This is not simply due to sorting. Look how long it takes to sort an array of 150,000 numbers:

```
random = (size) -> Math.floor Math.random()*size
random150000 = (random(150000) for number in [1..150000])

begin = new Date
random150000.sort()
end = new Date
console.log end - begin
# 50
```

Generating an array of 150,000 random numbers

50 milliseconds (time will vary on different computers)